Compiler

Book: Introduction to Theory of Computation
Authors: Anil Maheshwari, Michiel Smid
Edition: 2nd edition

Md. Nazmul Arefin
Assistant Lecturer
Dept. of Computer Science & Engineering
International Islamic University Chittagong

Automata – What is it?

- The term "Automata" is derived from the Greek word "αὐτόματα" which means "**self-acting**".
- An automaton (Automata in plural) is an abstract self-propelled computing device which follows a predetermined sequence of operations automatically.
- An automaton with a finite number of states is called a *Finite Automaton* (FA) or *Finite State Machine* (FSM).

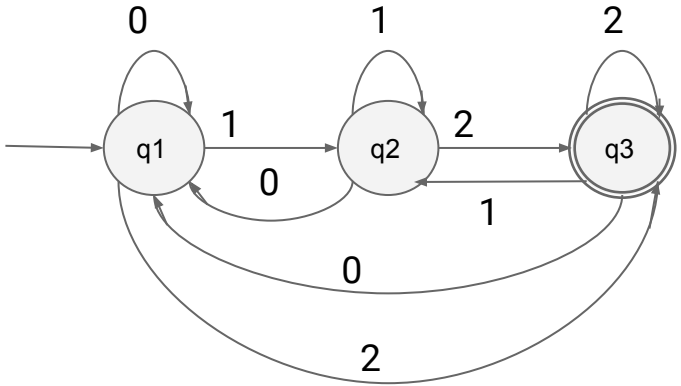
Automata – What is it?

Example:

- 2.1 @21 [1]
- 2.2 @23 [1]

Practice:

- Lift automaton



Transition system

	0	1	2
q0	q0	q1	q2
q1	q0	q1	q2
q2	q0	q1	q2

Transition table

Formal definition of a Finite Automaton

An automaton can be represented by a 5-tuple $(Q, \sum, \delta, q0, F)$, where –

- Q is a finite set of states.
- $\sum$ is a finite set of symbols, called the alphabet of the automaton.
- δ is the transition function.
- $q0$ is the initial state from where any input is processed ($q0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Related Terminologies

Alphabet

- Definition – An alphabet is any finite set of symbols.
- Example – ∑ = {a, b, c, d} is an alphabet set where ‘a’, ‘b’, ‘c’, and ‘d’ are symbols.

String

- Definition – A string is a finite sequence of symbols taken from ∑.
- Example – ‘cabcad’ is a valid string on the alphabet set ∑ = {a, b, c, d}

Length of a String

- Definition – It is the number of symbols present in a string. (Denoted by |S|).
- Examples –
  - If S = ‘cabcad’, |S|= 6
  - If |S|= 0, it is called an empty string (Denoted by λ or ε)

Related Terminologies

Kleene Star

- Definition – The Kleene star, $\sum^*$, is a unary operator on a set of symbols or strings, $\sum$, that gives the infinite set of all possible strings of all possible lengths over $\sum$ including λ.
- Representation – $\sum^* = \sum 0 \cup \sum 1 \cup \sum 2 \cup \ldots\ldots$ where $\sum p$ is the set of all possible strings of length p.
- Example – If $\sum$ = {a, b}, $\sum^*$ = {λ, a, b, aa, ab, ba, bb,.....}

Kleene Closure / Plus

- Definition – The set $\sum +$ is the infinite set of all possible strings of all possible lengths over $\sum$ excluding λ.
- Representation – $\sum + = \sum 1 \cup \sum 2 \cup \sum 3 \cup \ldots\ldots$
  $\sum + = \sum^* - \{ \lambda \}$
- Example – If $\sum$ = { a, b } , $\sum +$ = { a, b, aa, ab, ba, bb,.....}

Examples

- 2.2.1 @ 26 [2]
- 2.2.2 @ 28 [2]
- 2.2.3 @ 29 [2]

Nondeterministic Finite Automata

In NFA, for a particular input symbol, the machine can move to any combination of the states in the machine. In other words, the exact state to which the machine moves cannot be determined. Hence, it is called Non-deterministic Automaton.

As it has finite number of states, the machine is called Non-deterministic Finite Machine or Non-deterministic Finite Automaton.

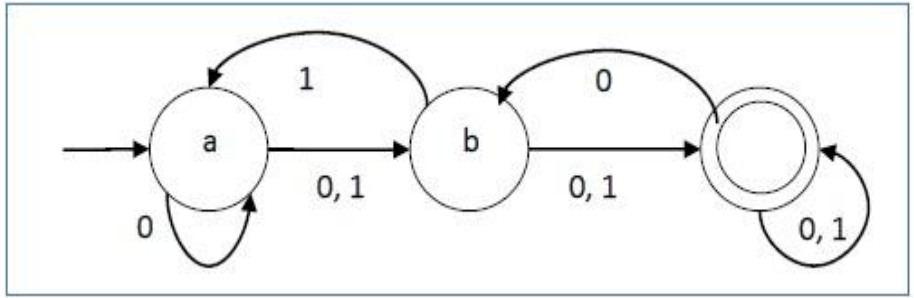
Nondeterministic Finite Automata

Let a non-deterministic finite automaton be →

- Q = {a, b, c}
- ∑ = {0, 1}
- q0 = {a}
- F = {c}

The transition function δ state diagram as shown below –

Present State	0	1
a	a, b	b
b	c	a, c
c	b, c	c



DFA vs NFA

DFA	NFA
The transition from a state is to a single particular next state for each input symbol. Hence it is called *deterministic*.	The transition from a state can be to multiple next states for each input symbol. Hence it is called *non-deterministic*.
Empty string transitions are not seen in DFA.	NFA permits empty string transitions.
Backtracking is allowed in DFA	In NFA, backtracking is not always possible.
Requires more space.	Requires less space.
A string is accepted by a DFA, if it transits to a final state.	A string is accepted by a NFA, if at least one of all possible transitions ends in a final state.

Table: Nfa vs Dfa [3]

Examples

- 2.4.1 @35 [2]
- 2.4.2 @37 [2]

NFA to DFA

Input – An NFA

Output – An equivalent DFA

Step 1 – Create state table from the given NFA.

Step 2 – Create a blank state table under possible input alphabets for the equivalent DFA.

Step 3 – Mark the start state of the DFA by q0 (Same as the NFA).

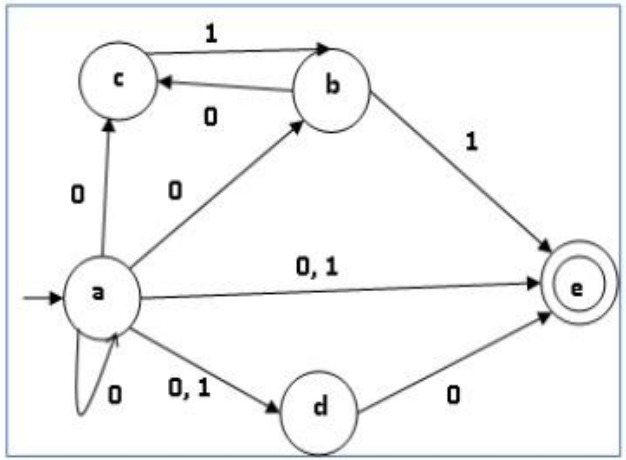
Step 4 – Find out the combination of States {Q0, Q1,… , Qn} for each possible input alphabet.

Step 5 – Each time we generate a new DFA state under the input alphabet columns, we have to apply step 4 again, otherwise go to step 6.

Step 6 – The states which contain any of the final states of the NFA are the final states of the equivalent DFA.

NFA to DFA

Example [3]:



NFA

q	δ(q,0)	δ(q,1)
a	{a,b,c,d,e}	{d,e}
b	{c}	{e}
c	∅	{b}
d	{e}	∅
e	∅	∅

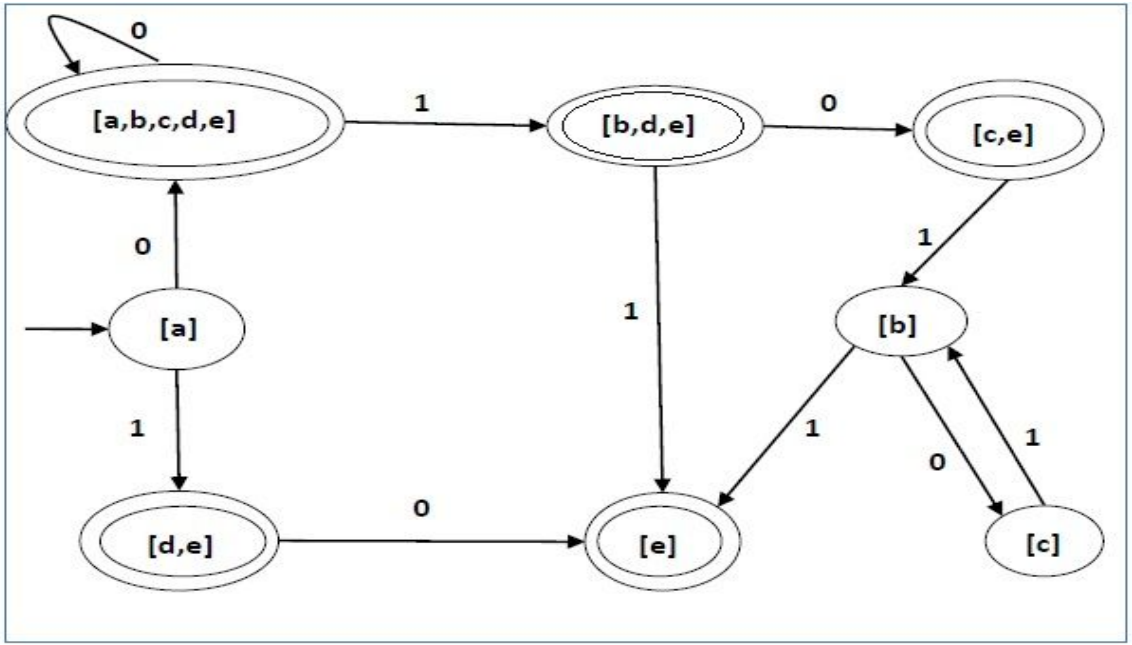
State table of NFA

q	δ(q,0)	δ(q,1)
[a]	[a,b,c,d,e]	[d,e]
[a,b,c,d,e]	[a,b,c,d,e]	[b,d,e]
[d,e]	[e]	∅
[b,d,e]	[c,e]	[e]
[e]	∅	∅
[c, e]	∅	[b]
[b]	[c]	[e]
[c]	∅	[b]

State table of DFA

NFA to DFA

Example [3]:



State diagram of DFA

NFA to DFA

Practice:

- 2.5.1 @44 [2]

Regular Expression

A Regular Expression can be recursively defined as follows –

- ε is a Regular Expression indicates the language containing an **empty string.** (L (ε) = {ε})
- φ is a Regular Expression denoting an **empty language**. (L (φ) = { })
- x is a Regular Expression where L = {x}
- If X is a Regular Expression denoting the language L(X) and Y is a Regular Expression denoting the language L(Y), then
  - X + Y is a Regular Expression corresponding to the language L(X) ∪ L(Y) where L(X+Y) = L(X) ∪ L(Y).
  - X . Y is a Regular Expression corresponding to the language L(X) . L(Y) where L(X.Y) = L(X) . L(Y)
  - R* is a Regular Expression corresponding to the language L(R*)where L(R*) = (L(R))*
- If we apply any of the rules several times from 1 to 5, they are Regular Expressions.

Regular Expression

Regular Expressions	Regular Set
(0 + 10*)	L = { 0, 1, 10, 100, 1000, 10000, ... }
(0*10*)	L = {1, 01, 10, 010, 0010, ...}

Table: Regular expression and languages [3]

Regular Expression

Regular Expressions	Regular Set
(0 + ε)(1 + ε)	L = {ε, 0, 1, 01}
(a+b)*	Set of strings of a's and b's of any length including the null string. So L = { ε, a, b, aa , ab , bb , ba, aaa.....}

Table: Regular expression and languages [3]

Regular Expression

Regular Expressions	Regular Set
(a+b)*abb	Set of strings of a's and b's ending with the string abb. So L = {abb, aabb, babb, aaabb, ababb, ...}
(11)*	Set consisting of even number of 1's including empty string, So L= {ε, 11, 1111, 111111, ...}

Table: Regular expression and languages [3]

Regular Expression

Regular Expressions	Regular Set
(aa)*(bb)*b	Set of strings consisting of even number of a's followed by odd number of b's , so L = {b, aab, aabbb, aabbbbb, aaaab, aaaabbb, ……}
(aa + ab + ba + bb)*	String of a's and b's of even length can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null, so L = {aa, ab, ba, bb, aaab, aaba, ……}

Table: Regular expression and languages [3]

Regular Expression to DFA

- See the provided sheet [3] & video.

Role of lexical analyzer

- See section **3.1** [1].

Token, Lexeme, Pattern

- Tokens are terminal symbols of the source language.

  e.g. *identifiers, numbers, keywords, punctuation symbols etc.*

- Lexemes are specific instance of a token. These are matched against pattern.

  Result = Result + Rate;

  Here, tokens are: identifier, operators, punctuation.

- Pattern is a **rule** describing all those lexemes that can represent a particular token in a source language.

Token, Lexeme, Pattern

Example:

12+10-15

- **Tokens**: Numbers, Operators
- **Lexemes**:
  Numbers: 12,10,15
  Operators: +,-
- **Pattern**:
  Numbers: [0-9]*
  (set of digits where it can be a single digit or followed by multiple digits)

References

[1] Compilers, principles, techniques, and tools. *Aho, Alfred V. II. Aho, Alfred V.*
[2] Theory of computation, *Anil Maheshwari & Michiel Smid*
[3] https://www.tutorialspoint.com/index.htm
[4] https://www.geeksforgeeks.org